

Phase 6: User Interface Development – Donation Tracking System for NGOs

1 Lightning App Builder

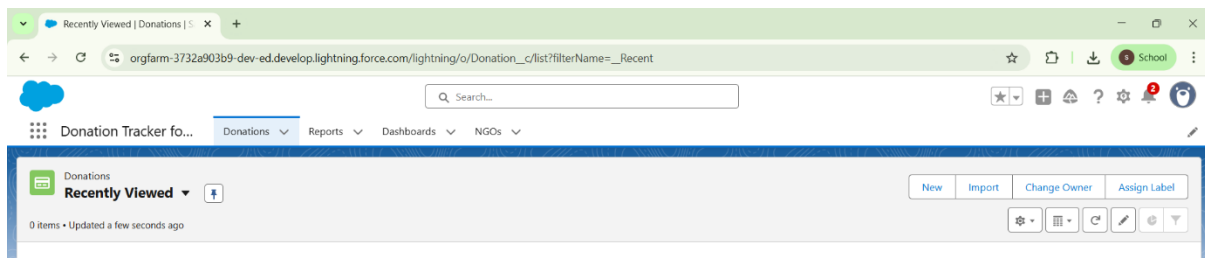
Use Case:

To give NGO administrators an easy way to manage donations and NGOs, we created a **custom Lightning App** using **Lightning App Builder**.

This app combines key tabs (Donations, NGOs, Reports, Dashboards) into one streamlined experience.

Steps:

- Go to **Setup → App Manager → Click New Lightning App**.
- Enter App Name: **Donation Tracker for NGOs**.
- Add utility items (optional).
- Under **Navigation Items**, add **Donations, NGOs, Reports, Dashboards**.
- Save and activate the app.



2 Record Pages

Use Case:

We built **custom record pages** for **Donation__c** and **NGO__c** objects.

This allows users to see donation details, related NGO information, phone numbers, receipts, and status on one page.

Steps / Procedure:

1. Go to **Setup → Lightning App Builder → New → Record Page**.
2. Select Object: **Donation__c** → Choose layout template.

3. Drag components: **Record Details**, **Related Lists**, and any custom LWCs.
4. Save and **Activate** the page (Org Default or App Default).
5. Repeat same for **NGO__c** object.
6. Open a Donation/NGO record to verify the new page layout.

3 Tabs

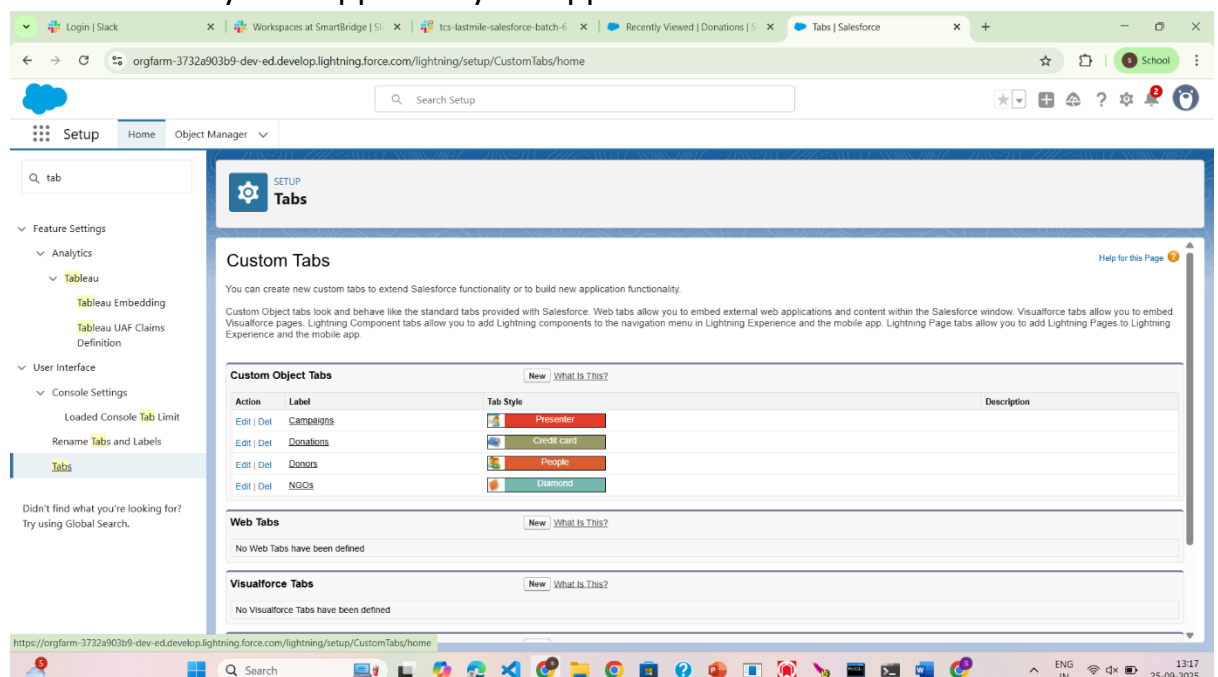
Use Case:

We added custom **Tabs** for Donations and NGOs to make navigation easy for end-users.

This allows one-click access to donation and NGO records without going to Object Manager every time.

Steps / Procedure:

1. Go to **Setup** → **Tabs** → **New** under Custom Object Tabs.
2. Select **Donation__c** and **NGO__c** objects.
3. Choose icons/colors → Save.
4. Go back to **App Manager** → **Edit your app** → **Navigation Items**.
5. Add **Donations & NGOs tabs** to Selected Items.
6. Save and verify tabs appear in your app.



4 Home Page Layouts

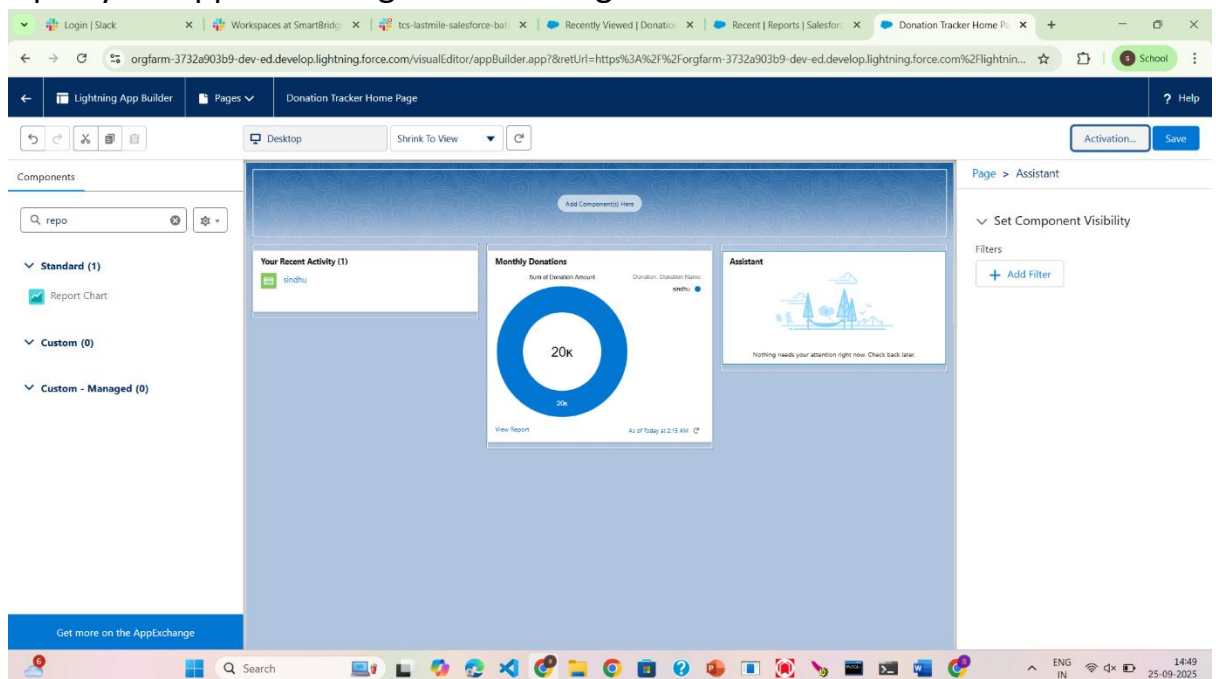
Use Case:

Provide a quick dashboard-style overview for NGO admins when they open the app. The **Home Page** shows:

- Recent Donations
- Key Reports (e.g., Donations by NGO)
- Quick links to create new records

Steps / Procedure:

1. Go to **Setup → Lightning App Builder → New → App Page / Home Page**.
2. Choose template: **Header + 3 Regions** (or your preference).
3. Drag components: **Recent Items, Report Chart, Assistant**.
4. Save and Activate as **App Default**.
5. Open your app Home Page to see changes.



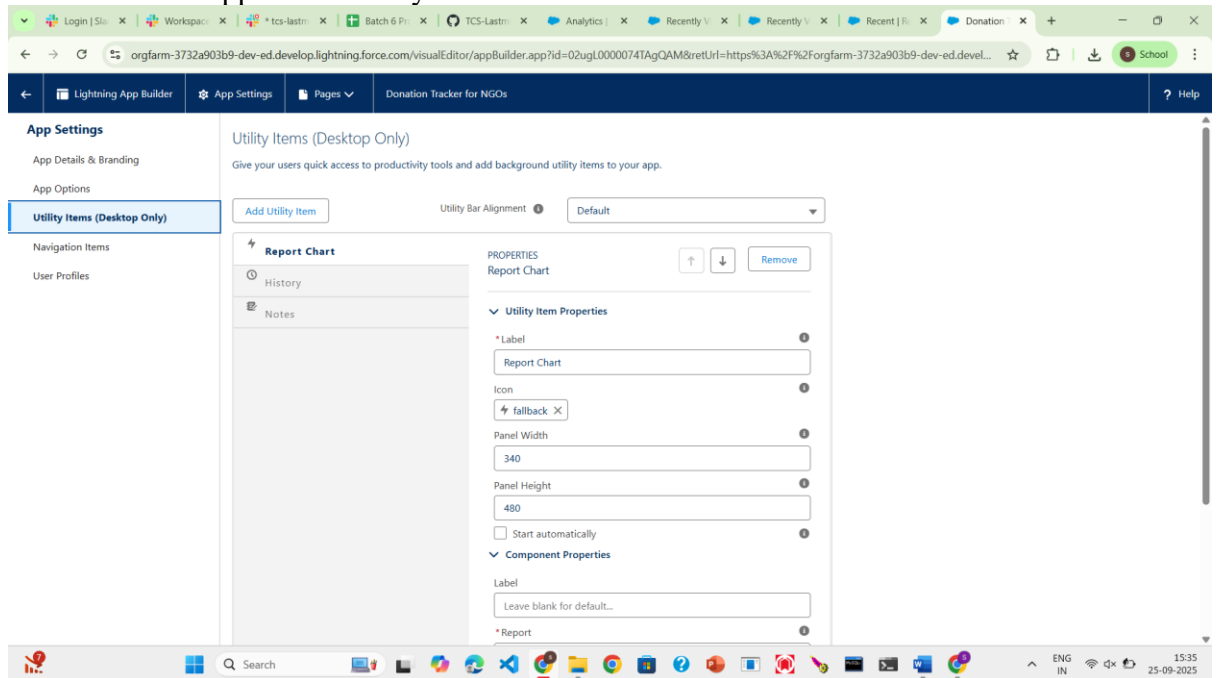
5 Utility Bar

Use Case:

The **Utility Bar** lets NGO admins access frequent tools quickly (e.g., Create New Donation, Check NGO Stats).

Steps / Procedure:

1. Go to **Setup** → **App Manager** → **Edit your App** → **Utility Items**.
2. Click **Add Utility Item**: choose **Notes**, **History**, or custom LWC.
3. Save & refresh app to see Utility Bar at bottom.



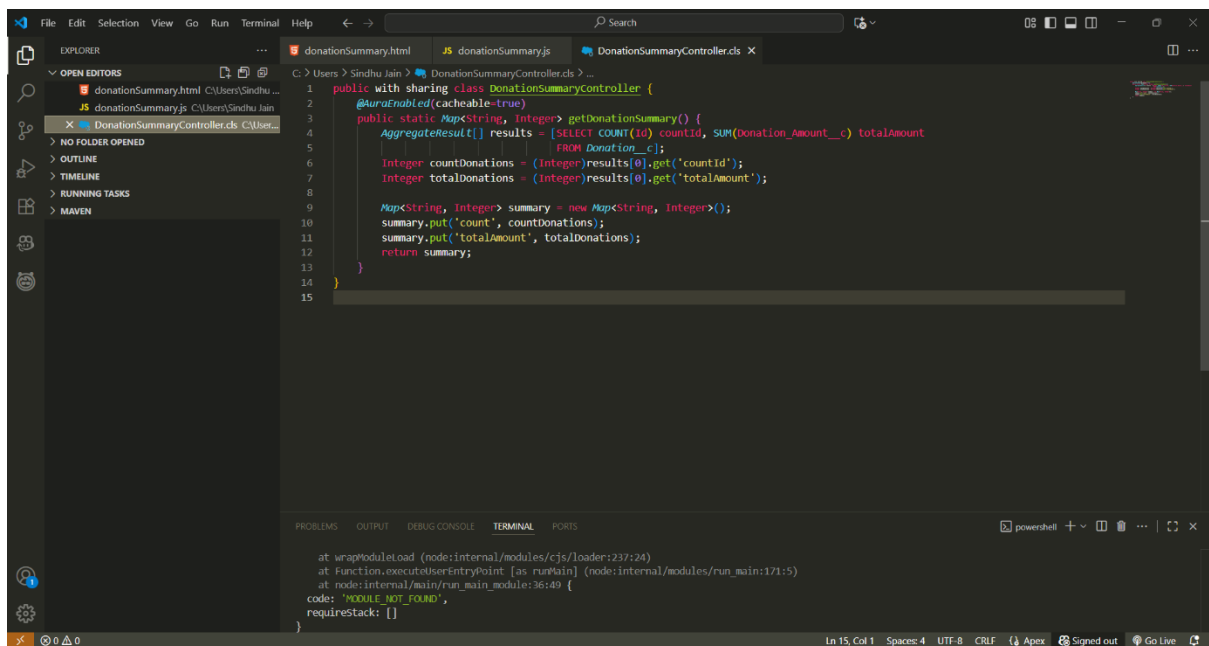
6 LWC (Lightning Web Components)

- **Use Case:**

We built a simple **Donation Summary LWC** to display the total donations received for an NGO. This adds real-time insights without needing reports.

Steps / Procedure:

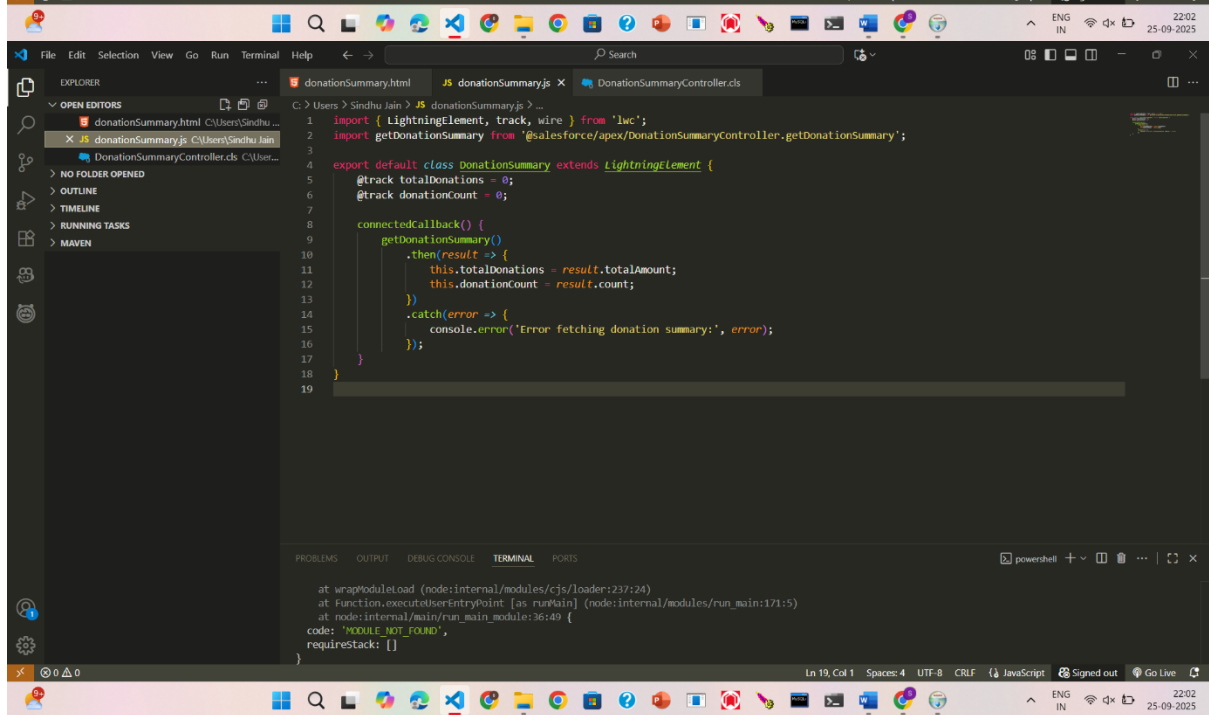
- In VS Code / Developer Console, create new LWC (e.g., donationSummary).
- In HTML file, design component layout. In JS file, fetch data using @wire or imperative Apex. give me full steps for this Deploy to org.
- In Lightning App Builder, drag the LWC onto the Donation Record Page.



```
1 public with sharing class DonationSummaryController {
2     @AuraEnabled(cacheable=true)
3     public static Map<String, Integer> getDonationSummary() {
4         AggregateResult[] results = [SELECT COUNT(id) countid, SUM(Donation_Amount__c) totalAmount
5                                     FROM Donation__c];
6         Integer countDonations = (Integer)results[0].get('countid');
7         Integer totalDonations = (Integer)results[0].get('totalAmount');
8
9         Map<String, Integer> summary = new Map<String, Integer>();
10        summary.put('count', countDonations);
11        summary.put('totalAmount', totalDonations);
12        return summary;
13    }
14 }
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

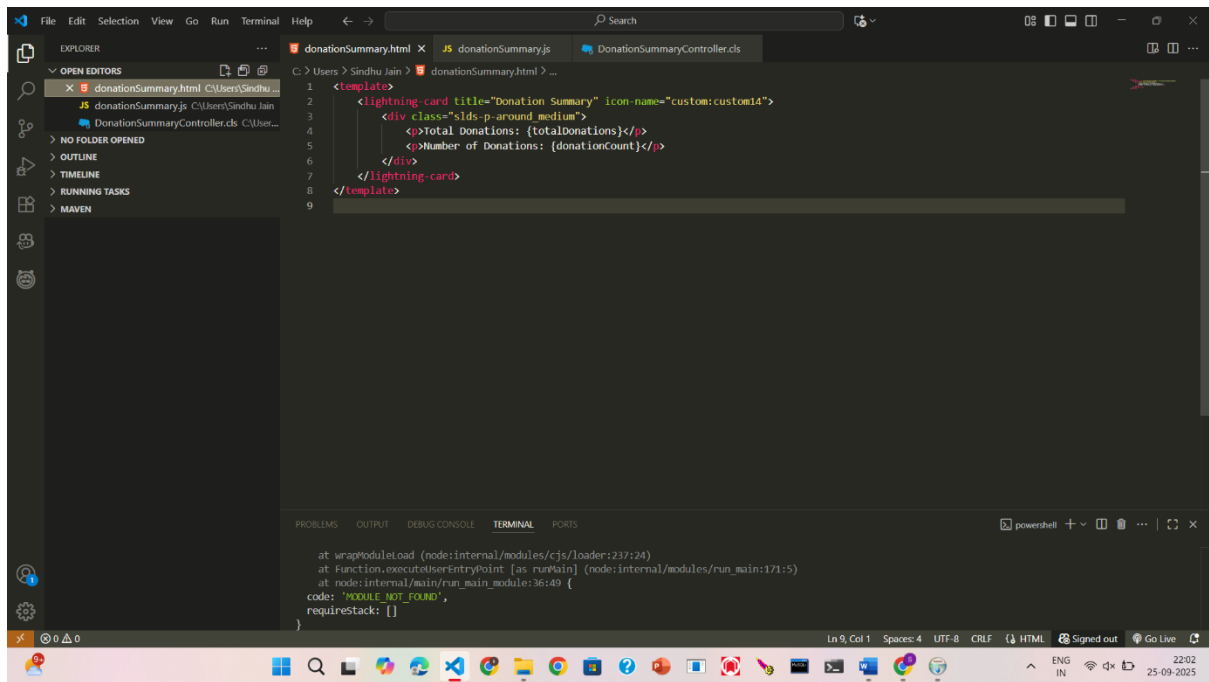
at wrapModuleLoad (node:internal/modules/cjs/loader:237:24)
at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:171:5)
at node:internal/main/run_main_module:36:49 {
 code: 'MODULE_NOT_FOUND',
 requireStack: []
}



```
1 import { LightningElement, track, wire } from 'lwc';
2 import getDonationSummary from '@salesforce/apex/DonationSummaryController.getDonationSummary';
3
4 export default class DonationSummary extends LightningElement {
5     @track totalDonations = 0;
6     @track donationCount = 0;
7
8     connectedCallback() {
9         getDonationSummary()
10            .then(result => {
11                this.totalDonations = result.totalAmount;
12                this.donationCount = result.count;
13            })
14            .catch(error => {
15                console.error('Error fetching donation summary:', error);
16            });
17     }
18 }
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

at wrapModuleLoad (node:internal/modules/cjs/loader:237:24)
at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:171:5)
at node:internal/main/run_main_module:36:49 {
 code: 'MODULE_NOT_FOUND',
 requireStack: []
}



7 Apex with LWC

- **Use Case:**

We connected LWC with **Apex** to fetch real-time donation data from Salesforce. This gives NGO admins immediate analytics.

Steps / Procedure:

1. Create Apex class: DonationSummaryController returning totals.
2. Annotate with **@AuraEnabled** for LWC to access.
3. Import method in LWC JS file.
4. Display returned values in LWC HTML.

